

Propuesta de solución PRA1

Nombre y apellidos del estudiante: Víctor Suesta Arribas

Correo electrónico UOC del estudiante: suave@uoc.edu

Aspectos generales

Este documento presenta una plantilla que deberéis utilizar para escribir vuestra solución. Por lo tanto, la solución a la actividad deberá proveerse a partir del documento actual.

El formato del archivo a entregar puede ser .pdf, .odt, .doc y .docx. Para otras opciones, por favor, contactar previamente con vuestro profesor colaborador.

El nombre del fichero debe contener el código de la asignatura, vuestro apellido y vuestro nombre, así como el número de actividad (PRA1). Por ejemplo, apellido1_nombre_nosql_pra1.docx.

Deberéis utilizar el siguiente tipo de letra para vuestras respuestas:

- Arial, tamaño 11 para el texto normal, y
- Courier New 8 para el código.

La actividad es individual y se fundamenta en el trabajo personal del estudiante. Se recomienda consultar la información sobre integridad académica y plagio del siguiente enlace (<https://campus.uoc.edu/estudiant/microsites/plagi/es/index.html>) para que tengáis clara la semántica del plagio y en qué condiciones se puede penalizar vuestro trabajo.

Para citar los materiales, en caso de ser necesario, hay que incluir las referencias de los documentos a citar, entendiendo documentos de cualquier tipo, al final del texto y citar los documentos en la explicación. Por ejemplo, si hablamos del modelo de grafo y queremos citar los materiales asociados, podríamos hacerlo de la siguiente manera:

Los modelos en grafo están compuestos de nodos, aristas, propiedades y etiquetas [1]...

Referencias:

[1] Jordi Conesa. Modelo en grafo [Vídeo].

<http://cimanet.uoc.edu/videos/bbdd/modelografo.html>

Ejercicio 1 (30%)

1.1 Ejercicio de modificación de datos (5%)

Explica qué ha pasado en cada comando de modificación de datos (insert, update) que se ha ejecutado en los pasos anteriores. Contesta las siguientes preguntas: ¿Por qué ha pasado esto? ¿Qué diferencias hay entre este comportamiento y el de una base de datos relacional o tradicional?

En este primer ejercicio he trabajado con la tabla `TESTING_AGGREGATE`, creada a partir del fichero `TESTING_AGGREGATE.csv`. Esta tabla tiene una clave primaria compuesta por `COLUMN1`, `COLUMN2` y `COLUMN3`, por lo que la combinación de estos tres valores identifica cada fila.

Después de cargar los datos iniciales, he comprobado el contenido de la tabla con la siguiente consulta:

```
SELECT *
FROM TESTING_AGGREGATE;
```

El resultado inicial obtenido ha sido:

```
columna1 | columna2 | columna3 | columna4
-----+-----+-----+-----
clave1_1 | clave2_1 | clave3_1 | valor1
clave1_2 | clave2_2 | clave3_2 | valor2
(2 rows)
```

A continuación, he ejecutado el INSERT indicado en el enunciado:

```
INSERT INTO TESTING_AGGREGATE (COLUMN1, COLUMN2, COLUMN3, COLUMN4)
VALUES ('clave1_2', 'clave2_2', 'clave3_2', 'valorNuevo');
```

Después de ejecutar esta sentencia, he vuelto a consultar la tabla:

```
SELECT *
FROM TESTING_AGGREGATE;
```

El resultado obtenido ha sido:

```
columna1 | columna2 | columna3 | columna4
-----+-----+-----+-----
clave1_1 | clave2_1 | clave3_1 | valor1
clave1_2 | clave2_2 | clave3_2 | valorNuevo
(2 rows)
```

En este caso observo que no se ha añadido una tercera fila. Lo que ha ocurrido es que ya existía una fila con la misma clave primaria, formada por los valores `clave1_2`, `clave2_2` y `clave3_2`. Por este motivo, Cassandra no ha insertado un nuevo registro independiente, sino que ha sustituido el valor anterior de `COLUMNA4`, cambiando `valor2` por `valorNuevo`.

Después he ejecutado el `UPDATE` indicado en el enunciado:

```
UPDATE TESTING_AGGREGATE
SET COLUMNA4 = 'valor3'
WHERE COLUMNA1 = 'clave1_3' AND COLUMNA2 = 'clave2_3' AND COLUMNA3 = 'clave3_3';
```

Al consultar de nuevo la tabla:

```
SELECT *
FROM TESTING_AGGREGATE;
```

he obtenido el siguiente resultado:

```
columna1 | columna2 | columna3 | columna4
-----+-----+-----+-----
clave1_1 | clave2_1 | clave3_1 | valor1
clave1_2 | clave2_2 | clave3_2 | valorNuevo
clave1_3 | clave2_3 | clave3_3 | valor3

(3 rows)
```

En este segundo caso, aunque he utilizado una sentencia `UPDATE`, Cassandra ha creado una nueva fila. Esto ocurre porque la clave primaria indicada en la cláusula `WHERE` no existía todavía en la tabla. Por tanto, Cassandra ha creado una fila nueva con los valores `clave1_3`, `clave2_3` y `clave3_3`, y ha asignado a `COLUMNA4` el valor `valor3`.

La explicación general es que Cassandra aplica un comportamiento de tipo `upsert` en las operaciones de escritura. Es decir, si la fila identificada por la clave primaria ya existe, la operación actualiza los valores correspondientes; si la fila no existe, la crea.

Desde mi punto de vista, esta es una diferencia importante respecto a una base de datos relacional tradicional. En una base de datos relacional, un `INSERT` con una clave primaria ya existente normalmente produciría un error por duplicidad, ya que la clave primaria debe ser única. Del mismo modo, un `UPDATE` sobre una fila que no existe no crearía un nuevo registro, sino que simplemente afectaría a cero filas. En cambio, en Cassandra las escrituras se interpretan a partir de la clave primaria indicada, lo que permite que un `INSERT` actúe como actualización y que un `UPDATE` actúe como inserción si la fila todavía no existe.

1.2 Consulta 1 (10%)

Tal y como hemos visto, ha habido algún problema en la carga de datos del fichero de viajes en bicicleta. La tabla TRIPS debería tener 124.954 registros, pero sólo tiene 43.049 registros. Explica por qué la inserción se ha realizado de forma errónea, haz una carga de datos que permita cargar los registros de forma correcta y pon un ejemplo de un registro que no se haya cargado en la primera ocasión y explica el porqué.

En este ejercicio he trabajado con el fichero viajes.csv, que contiene los viajes realizados por los usuarios. Primero he reproducido la carga inicial en una tabla TRIPS definida con ID_USER como única clave primaria:

```
DROP TABLE IF EXISTS TRIPS;

CREATE TABLE TRIPS (
  ID_USER TEXT,
  EMAIL TEXT,
  CREDITS DECIMAL,
  FROM_STATION INT,
  TO_STATION INT,
  BIKE TEXT,
  DATE TIMESTAMP,
  KILOMETERS DECIMAL,
  PRIMARY KEY (ID_USER));
```

Después he cargado el fichero viajes.csv:

```
COPY TRIPS (ID_USER, EMAIL, CREDITS, FROM_STATION, TO_STATION,
  BIKE, DATE, KILOMETERS) FROM
  '/home/student/Descargas/data/viajes.csv'
  WITH DELIMITER=',' AND HEADER=TRUE AND NULL='null';
```

La carga indica que se han procesado todos los registros del fichero:

```
Processed: 124954 rows; Rate:      5038 rows/s; Avg. rate:      7189 rows/s
124954 rows imported from 1 files in 17.383 seconds (0 skipped).
```

Sin embargo, al consultar el número real de filas almacenadas en la tabla, el resultado es mucho menor:

```
SELECT COUNT(*) FROM TRIPS;

## count

43049

(1 rows)

Warnings :
Aggregation query used without partition key
```

Por tanto, aunque Cassandra ha procesado 124.954 operaciones de escritura desde el fichero CSV, la tabla TRIPS solo contiene 43.049 filas distintas. El problema está en la

definición de la clave primaria. Al usar únicamente `ID_USER` como clave primaria, Cassandra solo puede almacenar una fila por cada usuario. Si un mismo usuario aparece en varios viajes, todos esos viajes comparten la misma clave primaria y se van sobrescribiendo entre sí.

Para corregir este problema, he creado una nueva tabla llamada `TRIPS_CORRECT`, añadiendo más columnas a la clave primaria para distinguir los distintos viajes de un mismo usuario:

```
DROP TABLE IF EXISTS TRIPS_CORRECT;

CREATE TABLE TRIPS_CORRECT (
  ID_USER TEXT,
  EMAIL TEXT,
  CREDITS DECIMAL,
  FROM_STATION INT,
  TO_STATION INT,
  BIKE TEXT,
  DATE TIMESTAMP,
  KILOMETERS DECIMAL,
  PRIMARY KEY (ID_USER, DATE, BIKE, FROM_STATION, TO_STATION));
```

En esta nueva tabla, `ID_USER` sigue siendo la clave de partición, pero se añaden `DATE`, `BIKE`, `FROM_STATION` y `TO_STATION` como claves de agrupamiento. De esta manera, diferentes viajes de un mismo usuario pueden almacenarse como filas distintas dentro de la misma partición.

A continuación, he cargado de nuevo el fichero `viajes.csv` en la tabla corregida:

```
COPY TRIPS_CORRECT (ID_USER, EMAIL, CREDITS, FROM_STATION, TO_STATION,
  BIKE, DATE, KILOMETERS) FROM
  '/home/student/Descargas/data/viajes.csv'
  WITH DELIMITER=',' AND HEADER=TRUE AND NULL='null';
```

La carga se ha realizado correctamente:

```
Processed: 124954 rows; Rate:      6602 rows/s; Avg. rate:   10748 rows/s
124954 rows imported from 1 files in 11.625 seconds (0 skipped).
```

Y al consultar el número de filas almacenadas, ahora sí se conservan todos los registros:

```
SELECT COUNT(*) FROM TRIPS_CORRECT;

## count

124954

(1 rows)

Warnings :
Aggregation query used without partition key
```

Para comprobar de forma concreta qué pasaba en la primera carga, he buscado un usuario que tuviera más de un viaje en el fichero CSV. Con el siguiente comando sobre el fichero viajes.csv:

```
awk -F',' 'NR>1 {count[$1]++} END {for (u in count) if (count[u]>1) {print u, count[u]; exit}}' viajes.csv
```

he obtenido el siguiente resultado:

72192326M 4

Esto indica que el usuario 72192326M aparece 4 veces en el fichero CSV, es decir, tiene 4 viajes registrados. Al consultar este usuario en la tabla corregida, se obtienen los 4 viajes:

```
SELECT ID_USER, DATE, BIKE, FROM_STATION, TO_STATION, CREDITS
FROM TRIPS_CORRECT
WHERE ID_USER = '72192326M';
```

id_user	date	bike	from_station	to_station	credits
72192326M	2024-12-11 18:55:52.000000+0000	UKXKJPXJQQ	58	7	0.62
72192326M	2024-12-18 15:24:58.000000+0000	7TFDKP4YXR	39	6	0.21
72192326M	2024-12-21 06:41:09.000000+0000	EZ919ETSJM	15	51	0.32
72192326M	2025-01-03 01:02:43.000000+0000	FD7D5YHRNV	56	32	0.4

(4 rows)

En cambio, al consultar el mismo usuario en la tabla inicial TRIPS, solo aparece una fila:

```
SELECT ID_USER, DATE, BIKE, FROM_STATION, TO_STATION, CREDITS
FROM TRIPS
WHERE ID_USER = '72192326M';
```

id_user	date	bike	from_station	to_station	credits
72192326M	2025-01-03 01:02:43.000000+0000	UKXKJPXJQQ	58	51	0.62

(1 rows)

Este ejemplo muestra claramente el problema de la primera tabla. Por ejemplo, el viaje del usuario 72192326M realizado el 2024-12-18 con la bicicleta 7TFDKP4YXR, desde la estación 39 hasta la estación 6, no aparece como una fila independiente en la tabla TRIPS. Lo mismo ocurre con otros viajes del mismo usuario. Todos esos registros compartían el mismo valor de ID_USER y, por tanto, se han ido sobrescribiendo en la misma fila.

Además, la fila resultante en `TRIPS` no representa de forma fiable uno de los viajes originales, ya que se observan valores procedentes de diferentes registros del mismo usuario. Esto se debe a que Cassandra ha ido actualizando columnas asociadas a la misma clave primaria.

En conclusión, la inserción inicial se ha realizado de forma errónea porque la clave primaria elegida no identificaba de forma única cada viaje, sino únicamente cada usuario. Como un usuario puede realizar varios viajes, era necesario incluir más atributos en la clave primaria. Con la tabla `TRIPS_CORRECT`, la combinación de `ID_USER`, `DATE`, `BIKE`, `FROM_STATION` y `TO_STATION` permite distinguir los distintos viajes y conservar correctamente los 124.954 registros del fichero.

1.3 Consulta 2 (25%)

Nos piden crear una consulta para analizar la disponibilidad de cascos en las bicicletas de la marca Orbea. La consulta debe devolver el número de bicicletas, la suma de cascos disponibles (helmets) y el total de kilómetros recorridos de las bicicletas de la marca Orbea agrupados por su estado.

En este ejercicio se pide analizar la disponibilidad de cascos en las bicicletas de la marca Orbea, agrupando los resultados por estado de la bicicleta. En concreto, la consulta debe devolver el número de bicicletas, la suma de cascos disponibles y el total de kilómetros recorridos para cada estado.

Para resolverlo, he creado una tabla específica orientada a esta consulta. Como en Cassandra el diseño debe partir del patrón de acceso, he definido `BRAND` como clave de partición, ya que la consulta filtra por la marca Orbea. Además, he incluido `STATUS` como clave de agrupamiento para poder agrupar los resultados por estado, e `ID` para identificar de forma única cada bicicleta dentro de cada grupo.

```
DROP TABLE IF EXISTS BIKES_BY_BRAND_STATUS;
```

```
CREATE TABLE BIKES_BY_BRAND_STATUS
(
  ID TEXT,
  BRAND TEXT,
  BATTERY_CHARGE DECIMAL,
  STATUS TEXT,
  HELMETS INT,
  EXTRA_FEATURES LIST<text>,
  DAMAGES LIST<text>,
  IS_CLEAN BOOLEAN,
  RIDES INT,
  KMS DECIMAL,
  LOCATION_TYPE TEXT,
  LOCATION_COORDINATES LIST<decimal>,
  PRIMARY KEY ((BRAND), STATUS, ID)
);
```

A continuación, he cargado el agregado `bikes.csv`, que contiene la información necesaria sobre las bicicletas:

```
COPY BIKES_BY_BRAND_STATUS (ID, BRAND, BATTERY_CHARGE, STATUS, HELMETS,
```

```
EXTRA_FEATURES, DAMAGES, IS_CLEAN, RIDES, KMS, LOCATION_TYPE,
LOCATION_COORDINATES) FROM
'/home/student/Descargas/data/bikes.csv'
WITH DELIMITER=';' AND HEADER=TRUE;
```

El resultado de la carga ha sido:

Using 1 child processes

```
Starting copy of bikes.bikes_by_brand_status with columns [id, brand, battery_charge,
status, helmets, extra_features, damages, is_clean, rides, kms, location_type,
location_coordinates].
Processed: 2000 rows; Rate: 2066 rows/s; Avg. rate: 3401 rows/s
2000 rows imported from 1 files in 0.588 seconds (0 skipped).
```

Después he comprobado que se habían cargado correctamente las 2000 bicicletas:

```
SELECT COUNT(*)
FROM BIKES_BY_BRAND_STATUS;

## count

2000

(1 rows)

Warnings :
Aggregation query used without partition key
```

Finalmente, he ejecutado la consulta solicitada:

```
SELECT STATUS, COUNT(*) AS num_bikes, SUM(HELMETS) AS total_helmets, SUM(KMS) AS total_kms
FROM BIKES_BY_BRAND_STATUS
WHERE BRAND = 'Orbea'
GROUP BY STATUS;
```

El resultado obtenido ha sido:

```
status | num_bikes | total_helmets | total_kms
-----+-----+-----+-----
free | 144 | 167 | 8608.836175172555331
moving | 128 | 151 | 7582.986817390795574
out | 232 | 287 | 14091.599613048982984
reserved | 134 | 160 | 8249.880660550304022

(4 rows)
```

Con esta consulta se obtiene la información pedida para las bicicletas de la marca Orbea, agrupada por su estado. La tabla creada permite resolver la consulta sin usar `ALLOW FILTERING`, ya que el filtro principal se realiza sobre la clave de partición `BRAND`. Además, `STATUS` se utiliza como clave de agrupamiento, lo que permite agrupar los resultados dentro de la partición correspondiente a la marca Orbea. Desde mi punto de vista, esta estructura encaja con el enfoque de Cassandra porque organiza los datos según la consulta que se

quiere responder, en lugar de intentar hacer una consulta libre sobre cualquier atributo de la tabla.

1.4 Consulta 3 (10%)

Supón que distintas ciudades han contratado a nuestra empresa para gestionar sus bicicletas. En consecuencia, se espera una cantidad de datos mucho mayor y, para poder gestionarlos eficientemente, se propone crear un sistema que reparta la carga de los datos de las bicicletas en 10 nodos de forma distribuida. Analiza si el agregado que has creado en la consulta anterior sería eficiente bajo estas premisas. En caso contrario, propón un nuevo agregado que se comporte de forma más eficiente. Justifica tu respuesta.

El agregado creado en la consulta anterior permite resolver correctamente la consulta pedida para la marca Orbea, ya que la clave de partición es `BRAND` y, por tanto, Cassandra puede localizar directamente las bicicletas de esa marca sin necesidad de usar `ALLOW FILTERING`.

La tabla utilizada en el apartado anterior era la siguiente:

```
CREATE TABLE BIKES_BY_BRAND_STATUS
(
  ID TEXT,
  BRAND TEXT,
  BATTERY_CHARGE DECIMAL,
  STATUS TEXT,
  HELMETS INT,
  EXTRA_FEATURES LIST<text>,
  DAMAGES LIST<text>,
  IS_CLEAN BOOLEAN,
  RIDES INT,
  KMS DECIMAL,
  LOCATION_TYPE TEXT,
  LOCATION_COORDINATES LIST<decimal>,
  PRIMARY KEY ((BRAND), STATUS, ID)
);
```

Sin embargo, considero que este diseño no sería eficiente si la empresa pasara a gestionar bicicletas de muchas ciudades y el volumen de datos aumentara mucho. El problema principal está en que la clave de partición es únicamente `BRAND`. Esto significa que todas las bicicletas de una misma marca, por ejemplo todas las bicicletas Orbea, quedarían agrupadas bajo la misma clave de partición.

En Cassandra, la clave de partición determina cómo se distribuyen los datos entre las particiones del clúster. Por tanto, si se utiliza solo `BRAND` como clave de partición, una marca muy frecuente podría generar una partición demasiado grande. En un escenario con 10 nodos, esto no favorecería un reparto equilibrado de la carga, porque las consultas sobre una marca concreta seguirían concentrándose en una única partición lógica.

Para mejorar este diseño, propondría crear un nuevo agregado añadiendo una columna de particionado artificial, por ejemplo `BUCKET`, con valores del 0 al 9. Este valor no vendría

directamente en el fichero CSV original, sino que habría que calcularlo antes de cargar los datos o durante el proceso de inserción. Por ejemplo, se podría obtener aplicando una función hash sobre el identificador de la bicicleta y tomando el resultado módulo 10. De esta forma, las bicicletas de una misma marca quedarían repartidas en 10 particiones diferentes.

Una posible tabla alternativa sería:

```
CREATE TABLE BIKES_BY_BRAND_BUCKET_STATUS
(
  ID TEXT,
  BRAND TEXT,
  BUCKET INT,
  BATTERY_CHARGE DECIMAL,
  STATUS TEXT,
  HELMETS INT,
  EXTRA_FEATURES LIST<text>,
  DAMAGES LIST<text>,
  IS_CLEAN BOOLEAN,
  RIDES INT,
  KMS DECIMAL,
  LOCATION_TYPE TEXT,
  LOCATION_COORDINATES LIST<decimal>,
  PRIMARY KEY ((BRAND, BUCKET), STATUS, ID)
);
```

Con este nuevo diseño, la consulta sobre la marca Orbea no se resolvería con una única partición, sino consultando los 10 buckets correspondientes. Por ejemplo, para uno de los buckets se podría ejecutar:

```
SELECT STATUS, COUNT(*) AS num_bikes, SUM(HELMETS) AS total_helmets, SUM(KMS) AS total_kms
FROM BIKES_BY_BRAND_BUCKET_STATUS
WHERE BRAND = 'Orbea' AND BUCKET = 0
GROUP BY STATUS;
```

La misma consulta se repetiría para `BUCKET = 1`, `BUCKET = 2`, ..., `BUCKET = 9`, y después la aplicación sumaría los resultados parciales por estado. Aunque esto obliga a realizar varias consultas y combinar los resultados fuera de Cassandra, permite repartir mejor los datos y evitar que todas las bicicletas de una misma marca se concentren en una única partición.

Desde mi punto de vista, este nuevo agregado sería más adecuado para un escenario distribuido con 10 nodos, porque reduce el riesgo de crear particiones demasiado grandes para marcas muy frecuentes. En Cassandra no solo importa poder responder una consulta, sino también que el diseño de la clave primaria permita distribuir bien los datos cuando el volumen crece. Por eso, para un escenario pequeño el agregado del apartado 1.3 es suficiente, pero para un sistema con muchas ciudades y más datos sería preferible introducir una estrategia de particionado adicional como `BUCKET`.

1.5 Consulta 4 (25%)

Nuestra empresa quiere realizar una limpieza general de las bicicletas que no están limpias. Como de momento no tiene capacidad para limpiarlas todas, empezará por las bicicletas de la marca Orbea que estén fuera de servicio (status = out). Para ello, nos piden crear una consulta para identificar las matrículas (ID) y la carga de batería de las bicicletas de la marca Orbea que estén fuera de servicio y no estén limpias (IS_CLEAN = false).

En este ejercicio se pide identificar las bicicletas de la marca Orbea que están fuera de servicio (status = out) y que, además, no están limpias (IS_CLEAN = false). Para cada bicicleta se debe mostrar su matrícula (ID) y su carga de batería (BATTERY_CHARGE).

Para resolver esta consulta he creado un agregado específico a partir del fichero bikes.csv. Como la consulta filtra por marca, estado y limpieza, he incluido estos tres campos en la clave de partición. De esta manera, Cassandra puede localizar directamente las bicicletas que cumplen las tres condiciones, sin necesidad de recorrer toda la tabla ni usar ALLOW FILTERING.

```
DROP TABLE IF EXISTS BIKES_BY_BRAND_STATUS_CLEAN;
```

```
CREATE TABLE BIKES_BY_BRAND_STATUS_CLEAN
(
  ID TEXT,
  BRAND TEXT,
  BATTERY_CHARGE DECIMAL,
  STATUS TEXT,
  HELMETS INT,
  EXTRA_FEATURES LIST<text>,
  DAMAGES LIST<text>,
  IS_CLEAN BOOLEAN,
  RIDES INT,
  KMS DECIMAL,
  LOCATION_TYPE TEXT,
  LOCATION_COORDINATES LIST<decimal>,
  PRIMARY KEY ((BRAND, STATUS, IS_CLEAN), ID)
);
```

Después he cargado el fichero bikes.csv en la tabla creada:

```
COPY BIKES_BY_BRAND_STATUS_CLEAN (ID, BRAND, BATTERY_CHARGE, STATUS, HELMETS,
EXTRA_FEATURES, DAMAGES, IS_CLEAN, RIDES, KMS, LOCATION_TYPE,
LOCATION_COORDINATES) FROM
'/home/student/Descargas/data/bikes.csv'
WITH DELIMITER=',' AND HEADER=TRUE;
```

El resultado de la carga ha sido:

```
Using 1 child processes
```

```
Starting copy of bikes.bikes_by_brand_status_clean with columns [id, brand, battery_charge,
status, helmets, extra_features, damages, is_clean, rides, kms, location_type,
location_coordinates].
```

Processed: 2000 rows; Rate: 2584 rows/s; Avg. rate: 3421 rows/s
2000 rows imported from 1 files in 0.585 seconds (0 skipped).

A continuación, he comprobado que se habían cargado correctamente las 2000 bicicletas:

```
SELECT COUNT(*)
FROM BIKES_BY_BRAND_STATUS_CLEAN;
```

```
## count
```

```
2000
```

```
(1 rows)
```

Warnings :

Aggregation query used without partition key

Finalmente, he ejecutado la consulta solicitada:

```
SELECT ID, BATTERY_CHARGE
FROM BIKES_BY_BRAND_STATUS_CLEAN
WHERE BRAND = 'Orbea'
AND STATUS = 'out'
AND IS_CLEAN = false;
```

El resultado obtenido ha sido:

id	battery_charge
0K401R1XVY	99.33
0PZJIJH82V	3.67
1YX4UMCJYG	0.0
24R8984099	0.0
3MXRCIAX2S	89.07
43VAJXUWDW	39.42
4YMU1NG23V	0.0
5RDK8ZWYDH	12.32
60JO4SAN1H	0.0
6UZMDUI5QB	0.0
8FMWNO7E9A	0.0
8KXXU4GHAL	63.98
90WIMZX8HN	0.0
9HJBGMPMEP	0.0
9ME3E89KTW	77.63
BX41WN2QXT	91.98
BXU5MQCD7N	0.0
EIFXVJ82Y5	0.0
GLA8JI9H95	48.13
HGMWDGSEZO	25.44
IDRW26X9FS	64.52
J7KE7358B9	80.2
JEVMCB54E7	0.0
KM81G1X2ZG	85.61
M14VQTXRZ5	0.0
MXD62P37QW	0.0
NC6QL26RUD	57.29
O611XF8XDX	28.83
OZ28R2OZ1U	0.0
PZGI6BBPZX	0.0
Q8T40TLKZU	62.79
Q9GYY1TIH0	0.0

```

QBZ6EQX4Q4 |          0.0
R6LSYES23C |         11.34
S7KA6OTP6M |          0.0
SJA11O542Y |          0.0
SQANOO3AFZ |          0.0
TVXOAH7QKE |         26.08
UMAM38CZS5 |          0.0
UWHSFDF7RO |          80.6
VJOMWVDJTO |          0.0
VMEJP3GB5H |         40.41
WSB8MVY1OG |         57.62
Z91XI0YP0Q |          0.0
ZIXZDE4LKY |         43.65

```

```
(45 rows)
```

El resultado muestra las 45 bicicletas de la marca Orbea que están fuera de servicio y no están limpias, junto con su nivel de batería. Desde mi punto de vista, el diseño de esta tabla es adecuado para la consulta planteada porque la clave de partición contiene exactamente los campos por los que se filtra: `BRAND`, `STATUS` e `IS_CLEAN`. Así se evita hacer una búsqueda general sobre todas las bicicletas y la consulta queda alineada con la forma de trabajar de Cassandra, donde normalmente se crea un agregado específico para cada patrón de acceso importante.

1.6 Consulta 5 (25%)

Para entender mejor el uso de cada bicicleta, nos piden una consulta que nos permita identificar, para cada bicicleta, los viajes realizados. La consulta deberá permitir, a partir del identificador de una bicicleta (`BIKE`), obtener los viajes que se han realizado con ella ordenados de más a menos reciente. Para cada viaje se deberá obtener el usuario, las estaciones de origen y destino, los kilómetros recorridos y los créditos consumidos.

Para demostrar el funcionamiento de la consulta, os animamos a que personalizéis la consulta para que obtenga la información de la bicicleta con identificador `EQN375DT02`.

Una vez se haya obtenido la solución, modifica la consulta para que devuelva los viajes de dos bicicletas distintas. ¿Qué pasa con la ordenación? ¿Continúan saliendo ordenados? Puedes utilizar las bicicletas con ID `'EQN375DT02'` y `'5FZ9GOGW97'` para comprobarlo.

En este ejercicio se pide crear una consulta que permita obtener, a partir del identificador de una bicicleta, los viajes realizados con ella ordenados de más a menos reciente. Para cada viaje deben mostrarse el usuario, las estaciones de origen y destino, los kilómetros recorridos y los créditos consumidos.

Para resolverlo, he creado un agregado específico a partir del fichero `viajes.csv`. Como la consulta se realiza a partir de la bicicleta, he usado `BIKE` como clave de partición. Además, he utilizado `DATE` como primera clave de agrupamiento y he definido el orden descendente, para que los viajes de cada bicicleta se almacenen ordenados de más reciente a menos reciente.

```
DROP TABLE IF EXISTS TRIPS_BY_BIKE;
```

```
CREATE TABLE TRIPS_BY_BIKE (
  ID_USER TEXT,
  EMAIL TEXT,
  CREDITS DECIMAL,
  FROM_STATION INT,
  TO_STATION INT,
  BIKE TEXT,
  DATE TIMESTAMP,
  KILOMETERS DECIMAL,
  PRIMARY KEY ((BIKE), DATE, ID_USER, FROM_STATION, TO_STATION)
)
WITH CLUSTERING ORDER BY (DATE DESC);
```

A continuación, he cargado el fichero viajes.csv en la tabla creada:

```
COPY TRIPS_BY_BIKE (ID_USER, EMAIL, CREDITS, FROM_STATION, TO_STATION,
  BIKE, DATE, KILOMETERS) FROM
  '/home/student/Descargas/data/viajes.csv'
WITH DELIMITER=',' AND HEADER=TRUE AND NULL='null';
```

El resultado de la carga ha sido:

```
Using 1 child processes
```

```
Starting copy of bikes.trips_by_bike with columns [id_user, email, credits, from_station,
to_station, bike, date, kilometers].
Processed: 124954 rows; Rate:      5966 rows/s; Avg. rate:    10649 rows/s
124954 rows imported from 1 files in 11.734 seconds (0 skipped).
```

Después he comprobado que se habían cargado todos los viajes:

```
SELECT COUNT(*)
FROM TRIPS_BY_BIKE;

## count

124954

(1 rows)

Warnings :
Aggregation query used without partition key
```

Para demostrar el funcionamiento de la consulta, primero he comprobado cuántos viajes tiene la bicicleta EQN375DT02:

```
SELECT COUNT(*)
FROM TRIPS_BY_BIKE
WHERE BIKE = 'EQN375DT02';

## count

51

(1 rows)
```

A continuación, he consultado los viajes realizados con esa bicicleta:

```
SELECT DATE, ID_USER, FROM_STATION, TO_STATION, KILOMETERS, CREDITS
FROM TRIPS_BY_BIKE
WHERE BIKE = 'EQN375DT02';
```

La consulta devuelve 51 viajes. En la salida se observa que aparecen ordenados de más reciente a menos reciente por el campo `DATE`. Muestro un extracto representativo del resultado:

date	id_user	from_station	to_station	kilometers
2025-11-11 14:41:12.000000+0000	35588501Y		41	52
4.647 0.7000000000000001				
2025-11-04 19:12:54.000000+0000	62538511Z		14	5
1.786 0.27				
2025-11-04 17:33:46.000000+0000	86966305D		52	13
1.296 0.19				
2025-11-04 12:27:32.000000+0000	67004734E		36	16
2.2640000000000002 0.34				
2025-11-03 12:10:42.000000+0000	98421918P		11	54
2.008 0.3000000000000004				
...				
2025-01-18 16:47:27.000000+0000	40006045P		25	43
4.084 0.61				
2024-12-30 14:28:47.000000+0000	45133260W		43	20
4.587 0.6900000000000001				
2024-12-20 17:07:02.000000+0000	63032208I		37	40
1.5670000000000002 0.24				
2024-12-17 14:47:30.000000+0000	36847578B		23	41
3.622 0.54				

(51 rows)

En este caso, la ordenación funciona correctamente porque todos los viajes pertenecen a la misma clave de partición, `BIKE = 'EQN375DT02'`. Dentro de esa partición, Cassandra almacena las filas ordenadas por `DATE DESC`, tal como se ha definido en la tabla.

Después he modificado la consulta para obtener los viajes de dos bicicletas distintas:

```
SELECT BIKE, DATE, ID_USER, FROM_STATION, TO_STATION, KILOMETERS, CREDITS
FROM TRIPS_BY_BIKE
WHERE BIKE IN ('EQN375DT02', '5FZ9GOGW97');
```

En mi caso, la consulta devuelve 100 filas en total. En el resultado se observa que primero aparecen los viajes de una bicicleta y después los de la otra. Muestro un extracto donde se ve este comportamiento:

bike	date	id_user	from_station	to_station	kilometers
5FZ9GOGW97	2025-11-07 13:44:33.000000+0000	80936058T		54	49
3.057 0.46					

5FZ9GOGW97		2025-11-03 09:02:16.000000+0000		55414327Y		11		22	
1.358		0.2							
5FZ9GOGW97		2025-10-26 11:51:02.000000+0000		11987480Y		13		39	
3.688		0.55							
...									
5FZ9GOGW97		2024-12-06 11:05:35.000000+0000		08049034E		14		27	
5.033		0.75							
EQN375DT02		2025-11-11 14:41:12.000000+0000		35588501Y		41		52	
4.647		0.70							
EQN375DT02		2025-11-04 19:12:54.000000+0000		62538511Z		14		5	
1.786		0.27							
EQN375DT02		2025-11-04 17:33:46.000000+0000		86966305D		52		13	
1.296		0.19							

Al consultar dos bicicletas distintas, la ordenación deja de ser global. Es decir, Cassandra no mezcla todos los viajes de ambas bicicletas para devolver una única lista ordenada por fecha. Lo que hace es devolver los datos por particiones: dentro de cada bicicleta sí se respeta el orden descendente por `DATE`, pero entre bicicletas distintas no hay una ordenación global.

Esto se ve claramente porque aparecen viajes de `5FZ9GOGW97` hasta fechas de diciembre de 2024 y, después, empiezan viajes de `EQN375DT02` con fechas de noviembre de 2025. Si la ordenación fuera global, esos viajes de noviembre de 2025 deberían aparecer antes que los de diciembre de 2024.

Por tanto, considero que el diseño de la tabla es adecuado para la consulta principal del enunciado, donde se parte de una única bicicleta. Sin embargo, cuando se consultan varias bicicletas con `IN`, la ordenación solo se mantiene dentro de cada partición individual, no entre todas las particiones consultadas conjuntamente. Para obtener una ordenación global entre varias bicicletas, habría que combinar y ordenar los resultados posteriormente desde la aplicación.

Ejercicio 2 (35%)

Consulta 1 (10%)

Muestra por pantalla el recuento de planetas que aparecen en la película 'Revenge of the Sith', de forma que, en la primera columna, aparezca el número de planetas y, en la segunda, una lista con los nombres de dichos planetas.

Para esta primera consulta se pide obtener el número de planetas que aparecen en la película Revenge of the Sith y, además, mostrar en una segunda columna la lista con los nombres de esos planetas.

He partido de los nodos `Planet` y de la relación `APPEARS_IN`, que conecta los planetas con las películas en las que aparecen. Después filtro la película por su nombre y uso `DISTINCT` para evitar contar dos veces un mismo planeta si hubiera más de una relación asociada.

```
MATCH (p:Planet)-[:APPEARS_IN]->(f:Film {name: 'Revenge of the Sith'})
WITH DISTINCT p
ORDER BY p.name
RETURN count(p) AS num_planets,
collect(p.name) AS planets;
```

El resultado obtenido ha sido:

```
num_planets | planets
-----+-----
5           | ["Alderaan", "Coruscant", "Kashyyyk", "Naboo", "Tatooine"]
```

Por tanto, observo que en la película Revenge of the Sith aparecen 5 planetas distintos. La lista obtenida es Alderaan, Coruscant, Kashyyyk, Naboo y Tatooine. Considero que la consulta responde directamente al enunciado porque devuelve en la primera columna el recuento total de planetas y en la segunda columna la lista con sus nombres.

Consulta 2 (15%)

Muestra por pantalla todas aquellas organizaciones que han sido lideradas, exactamente, por tres personajes. En la primera columna debe aparecer el nombre de la organización, en la segunda la cantidad de líderes que la han dirigido y, finalmente, en la tercera columna, una lista con los nombres de dichos líderes. Se pide que aparezcan ordenadas alfabéticamente por el nombre de las organizaciones.

Para esta segunda consulta se pide obtener las organizaciones que han sido lideradas exactamente por tres personajes. El resultado debe mostrar el nombre de la organización, el número de líderes y la lista con los nombres de dichos líderes. Además, las organizaciones deben aparecer ordenadas alfabéticamente por su nombre.

En este caso he partido de los nodos Organization y Character, conectados mediante la relación LEADER_OF. Agrupo por organización y recojo los nombres distintos de los personajes que han liderado cada una de ellas. Después filtro únicamente aquellas organizaciones cuya lista de líderes tiene tamaño 3.

```
MATCH (o:Organization)<-[:LEADER_OF]-(c:Character)
WITH o, collect(DISTINCT c.name) AS leaders
WHERE size(leaders) = 3
RETURN o.name AS organization,
size(leaders) AS num_leaders,
leaders AS leaders
ORDER BY organization;
```

El resultado obtenido ha sido:

organization	num_leaders	leaders
Jedi Order	3	["Anakin Skywalker", "Obi-Wan Kenobi", "Luke Skywalker"]
Rebel Alliance	3	["Lando Calrissian", "Luke Skywalker", "Han Solo"]
Resistance	3	["Poe Dameron", "Rey", "Finn"]
Sith Order	3	["Darth Maul", "Darth Vader", "Darth Tyranus"]

A partir del resultado, observo que hay cuatro organizaciones que cumplen la condición de haber sido lideradas exactamente por tres personajes: Jedi Order, Rebel Alliance, Resistance y Sith Order. La consulta responde al enunciado porque devuelve las tres columnas solicitadas y filtra explícitamente mediante size(leaders) = 3, quedándose solo con las organizaciones que tienen exactamente tres líderes distintos. Además, el resultado queda ordenado alfabéticamente por el nombre de la organización.

Consulta 3 (15%)

Se desea obtener el listado de los fabricantes de vehículos que han producido modelos que aparecen en más de una película. La consulta debe mostrar, en la primera columna, el nombre del fabricante, en la segunda el número de películas distintas en las que aparecen sus vehículos, en la tercera, una lista con los nombres de dichas películas y, en la última, una lista con los nombres, sin repetirlos, de los vehículos en cuestión. Se solicita ordenar los resultados de mayor a menor número de apariciones.

Para esta tercera consulta se pide obtener los fabricantes de vehículos que han producido modelos que aparecen en más de una película. El resultado debe mostrar el nombre del fabricante, el número de películas distintas en las que aparecen sus vehículos, la lista de películas y la lista de vehículos, evitando duplicados y ordenando de mayor a menor número de películas distintas.

En este caso he partido de los nodos Manufacturer, Vehicle y Film. La relación MANUFACTURED_BY conecta los vehículos con sus fabricantes, y la relación APPEARS_IN conecta los vehículos con las películas en las que aparecen. Primero calculo, para cada vehículo, la

lista de películas distintas en las que aparece. Después filtro únicamente aquellos vehículos que aparecen en más de una película y, finalmente, agrupo por fabricante.

```
MATCH (m:Manufacturer)<-[:MANUFACTURED_BY]-(v:Vehicle)-[:APPEARS_IN]->(f:Film)
WITH m, v, collect(DISTINCT f.name) AS vehicle_films
WHERE size(vehicle_films) > 1
WITH m, collect(DISTINCT v.name) AS vehicles, collect(vehicle_films) AS film_lists
UNWIND film_lists AS film_list
UNWIND film_list AS film_name
WITH m, vehicles, collect(DISTINCT film_name) AS films
RETURN m.name AS manufacturer,
size(films) AS num_films,
films AS films,
vehicles AS vehicles
ORDER BY num_films DESC, manufacturer;
```

El resultado obtenido ha sido:

```
manufacturer      | num_films | films |
vehicles
-----+-----+-----
---
Kuat Systems Engineering | 2 | ["Attack of the Clones", "The Empire Strikes Back"] |
["Slave 1"]
```

A partir del resultado, observo que el único fabricante que cumple la condición es Kuat Systems Engineering. El vehículo identificado es Slave 1, que aparece en dos películas distintas: Attack of the Clones y The Empire Strikes Back.

Considero que la consulta responde al enunciado porque filtra únicamente los vehículos con aparición en más de una película, agrupa posteriormente por fabricante y muestra tanto el número de películas distintas como las listas solicitadas. Además, se usa `DISTINCT` para evitar repetir nombres de películas o de vehículos, y el resultado queda ordenado de mayor a menor número de películas distintas.

Consulta 4 (20%)

Queremos obtener un listado de las naves, y los personajes que las pilotaron, que aparecen en las películas de la saga cuyo título contenga la palabra 'Empire'. Por ello, para cada resultado, se debe mostrar el nombre de la nave, el nombre del personaje que la pilotó, y el título de la película. Los resultados estarán ordenados ascendentemente por el nombre de la nave, posteriormente, si se repite, por el nombre del personaje y, de coincidir estos dos primeros campos, por el título de la película.

Para esta cuarta consulta se pide obtener las naves y los personajes que las pilotaron en aquellas películas de la saga cuyo título contiene la palabra Empire. El resultado debe mostrar el nombre de la nave, el nombre del personaje que la pilotó y el título de la película. Además, los resultados deben aparecer ordenados ascendentemente por el nombre de la nave, posteriormente por el nombre del personaje y, si coinciden los dos primeros campos, por el título de la película.

En este caso he partido de los nodos Character, Starship y Film. La relación `PILOTED` conecta los personajes con las naves que han pilotado, y la relación `APPEARS_IN` conecta las naves con las películas en las que aparecen. Después filtro las películas cuyo nombre contiene la palabra Empire.

```
MATCH (c:Character)-[:PILOTED]->(s:Starship)-[:APPEARS_IN]->(f:Film)
WHERE f.name CONTAINS 'Empire'
RETURN s.name AS starship,
       c.name AS character,
       f.name AS film
ORDER BY starship ASC, character ASC, film ASC;
```

El resultado obtenido ha sido:

starship	character	film
-----	-----	-----
Executor	Darth Vader	The Empire Strikes Back
Millennium Falcon	Chewbacca	The Empire Strikes Back
Millennium Falcon	Han Solo	The Empire Strikes Back
Outrider	Dash Rendar	Shadows of the Empire
Slave 1	Boba Fett	The Empire Strikes Back
Slave II	Boba Fett	The Empire Strikes Back
Snowspeeder	Luke Skywalker	The Empire Strikes Back
X-wing	Luke Skywalker	The Empire Strikes Back

A partir del resultado, observo que la consulta recupera 8 combinaciones de nave, personaje y película. La mayoría corresponden a The Empire Strikes Back, aunque también aparece Shadows of the Empire, ya que su título también contiene la palabra Empire.

Considero que la consulta responde correctamente al enunciado porque identifica las películas filtradas por el título, obtiene las naves que aparecen en ellas y recupera los personajes que las pilotaron. Además, el `ORDER BY` garantiza el orden solicitado: primero por el nombre de la nave, después por el nombre del personaje y, finalmente, por el título de la película.

Consulta 5 (20%)

Se desea obtener un listado de los personajes de la especie 'Human' que han pilotado naves clasificadas como 'Starfighter', y en qué películas aparecieron esas naves. Para ello, muestra, en la primera columna, el nombre de los personajes, ordenados alfabéticamente y, en la segunda, la lista de películas en que participó dicho personaje pilotando ese tipo de naves. Ordenar, descendientemente, por el número de películas de esa lista.

Para esta quinta consulta se pide obtener los personajes de especie Human que han pilotado naves clasificadas como Starfighter. El resultado debe mostrar, en la primera columna, el nombre del personaje y, en la segunda, la lista de películas en las que aparecen esas naves. Además, el resultado debe ordenarse descendientemente por el número de películas de dicha lista.

En este caso he partido de los nodos Character, Species, Starship, Class y Film. Primero filtro los personajes cuya especie es Human. Después busco las naves que han pilotado y restrinjo el resultado a aquellas cuya clase es Starfighter. Finalmente, recojo las películas distintas en las que aparecen esas naves. Para poder ordenar el resultado, calculo el tamaño de la lista de películas mediante una variable auxiliar, aunque no la muestro en la salida final porque el enunciado solo pide dos columnas.

```
MATCH (c:Character)-[:IS_OF_SPECIE]->(sp:Species {name: 'Human'})
MATCH (c)-[:PILOTED]->(s:Starship)-[:STARSHIP_CLASS]->(cl:Class {name: 'Starfighter'})
MATCH (s)-[:APPEARS_IN]->(f:Film)
WITH c.name AS character, collect(DISTINCT f.name) AS films
WITH character, films, size(films) AS num_films
RETURN character,
films
ORDER BY num_films DESC, character ASC;
```

El resultado obtenido ha sido:

character	films
Luke Skywalker	["The Force Awakens", "Return of the Jedi", "A New Hope", "The Last Jedi", "The Rise of Skywalker", "The Empire Strikes Back"]
Anakin Skywalker	["Attack of the Clones", "The Phantom Menace"]
Darth Vader	["A New Hope"]

A partir del resultado, observo que aparecen tres personajes humanos que han pilotado naves de clase Starfighter: Luke Skywalker, Anakin Skywalker y Darth Vader. El personaje con más películas asociadas es Luke Skywalker, seguido de Anakin Skywalker y Darth Vader.

Considero que la consulta responde correctamente al enunciado porque filtra primero los personajes de especie Human, restringe las naves a la clase Starfighter y agrupa las películas por personaje. Además, aunque el número de películas no se muestra como columna, sí se utiliza para ordenar el resultado de forma descendente, tal como se solicita.

Consulta 6 (20%)

Queremos analizar la relación entre personajes y vehículos, centrándonos únicamente en aquellos personajes que han pilotado al menos un vehículo. El objetivo final es clasificar a cada personaje según el tipo de vehículos que ha conducido, utilizando criterios basados en palabras clave presentes en los nombres de dichos vehículos.

Concretamente, un personaje se considerará **‘Conductor Militar’** si ha pilotado, al menos, un vehículo cuyo nombre contenga el término *‘speeder’* como parte de su nombre (no es necesario que el término forme una palabra), asociando este tipo de vehículo a maniobras rápidas o misiones de combate. Por otro lado, si no cumple la condición anterior pero ha pilotado algún vehículo cuyo nombre contenga el término *‘cruiser’*, se clasificará como **‘Conductor Civil’**, identificando así a quienes operan vehículos de transporte más grandes o no directamente militares. Es importante remarcar que solo se incluirán en el resultado los personajes que encajen en alguna de estas dos categorías.

Así pues, el resultado de la consulta debe mostrar el nombre del personaje, su categoría asignada (Conductor Militar o Conductor Civil), el número total de vehículos que ha pilotado y la lista de nombres de dichos vehículos. Para facilitar el análisis, mostrará primero los conductores militares y posteriormente los civiles, priorizando en cada categoría aquellos que tengan un número total de vehículos pilotados mayor.

Para esta sexta consulta se pide clasificar a los personajes que han pilotado al menos un elemento pilotable, considerando tanto vehículos como naves. La clasificación se realiza a partir de palabras clave presentes en los nombres de dichos elementos. En concreto, un personaje se considera Conductor Militar si ha pilotado al menos un vehículo o nave cuyo nombre contiene el término *speeder*. Si no cumple esta condición, pero ha pilotado algún elemento cuyo nombre contiene el término *cruiser*, se clasifica como Conductor Civil.

En esta consulta he partido de los nodos Character y de los elementos que han sido pilotados por ellos mediante la relación `PILOTED`. Como en el grafo los elementos pilotables pueden estar representados tanto como Vehicle como Starship, filtro ambos tipos de nodos. Después recojo la lista de nombres distintos pilotados por cada personaje y compruebo si alguno de esos nombres contiene las palabras clave *speeder* o *cruiser*, ignorando mayúsculas y minúsculas mediante `toLowerCase()`.

```
MATCH (c:Character)-[:PILOTED]->(v)
WHERE v:Vehicle OR v:Starship
WITH c, collect(DISTINCT v.name) AS vehicles
WITH c, vehicles,
any(vehicle IN vehicles WHERE toLower(vehicle) CONTAINS 'speeder') AS has_speeder,
any(vehicle IN vehicles WHERE toLower(vehicle) CONTAINS 'cruiser') AS has_cruiser
WITH c, vehicles,
CASE
WHEN has_speeder THEN 'Conductor Militar'
WHEN has_cruiser THEN 'Conductor Civil'
END AS category
WHERE category IS NOT NULL
RETURN c.name AS character,
category AS category,
size(vehicles) AS total_vehicles,
```

```
vehicles AS vehicles
ORDER BY CASE category
WHEN 'Conductor Militar' THEN 0
ELSE 1
END,
total_vehicles DESC,
character ASC;
```

El resultado obtenido ha sido:

character	category	total_vehicles	vehicles
Darth Maul	Conductor Militar	3	["Sith Speeder", "Scimitar", "Sith Infiltrator"]
Luke Skywalker	Conductor Militar	3	["X-34 Landspeeder", "Snowspeeder", "X-wing"]
Scout Trooper	Conductor Militar	2	["Speeder Bike", "Imperial Speeder Bike"]
Wedge Antilles	Conductor Militar	1	["Snowspeeder"]
Queen Amidala	Conductor Civil	3	["Naboo Royal Starship", "Naboo Royal Cruiser", "J-type 327 Nubian"]

A partir del resultado, observo que aparecen cuatro personajes clasificados como Conductor Militar: Darth Maul, Luke Skywalker, Scout Trooper y Wedge Antilles. Todos ellos han pilotado al menos un elemento cuyo nombre contiene speeder. También aparece Queen Amidala como Conductor Civil, ya que entre los elementos que ha pilotado se encuentra Naboo Royal Cruiser.

Considero que la consulta responde correctamente al enunciado porque solo incluye personajes que encajan en alguna de las dos categorías solicitadas. Además, muestra el nombre del personaje, la categoría asignada, el número total de elementos pilotados y la lista de dichos elementos. La clasificación también respeta la prioridad indicada: si un personaje cumple la condición de speeder, se clasifica como Conductor Militar; solo en caso contrario se comprueba si cumple la condición de cruiser. Finalmente, el resultado queda ordenado primero por categoría, mostrando antes los conductores militares, y dentro de cada categoría se prioriza a los personajes con mayor número total de elementos pilotados.

Ejercicio 3 (35%)

Consulta 1 (10%)

Se desea saber qué usuarios menores de edad (es decir, los que tienen una edad menor de 18 años) no han usado el servicio nunca (i.e. 0 días de uso) y al mismo tiempo tienen un crédito mayor de 4 unidades. Ordenar los usuarios en orden decreciente de crédito. Mostrar solamente el email del usuario (**NO** incluir el DNI), la edad, el día de uso del servicio y los créditos.

Para esta primera consulta de MongoDB se pide obtener los usuarios menores de edad que no han usado nunca el servicio y que, además, tienen más de 4 créditos disponibles. El resultado debe ordenarse de forma decreciente por créditos y debe mostrar únicamente el correo electrónico, la edad, los días de uso y los créditos, sin incluir el DNI del usuario.

Antes de ejecutar la consulta, he comprobado que estaba trabajando en la base de datos correcta:

```
db
```

El resultado ha sido:

```
bikes_app
```

La consulta ejecutada ha sido la siguiente:

```
db.users.find(
{
  age: { $lt: 18 },
  daysUsed: 0,
  credits: { $gt: 4 }
},
{
  _id: 0,
  email: 1,
  age: 1,
  daysUsed: 1,
  credits: 1
})
.sort({ credits: -1 }).pretty()
```

El resultado obtenido ha sido:

```
{
  "email" : "mariarosagutierrezarias@hotmail.com",
  "age" : 14,
  "daysUsed" : 0,
  "credits" : 4.75
}
{
  "email" : "c.alonsoleon@outlook.com",
  "age" : 15,
```

```

    "daysUsed" : 0,
    "credits" : 4.74
  }
  {
    "email" : "f.sanchezmarin@outlook.com",
    "age" : 17,
    "daysUsed" : 0,
    "credits" : 4.56
  }
  {
    "email" : "miguelangelcruzcalvo@gmail.com",
    "age" : 14,
    "daysUsed" : 0,
    "credits" : 4.55
  }
  {
    "email" : "oscarvargasdelgado@gmail.com",
    "age" : 15,
    "daysUsed" : 0,
    "credits" : 4.29
  }
}

```

En esta consulta he usado tres condiciones en el filtro: `age: { $lt: 18 }` para seleccionar menores de edad, `daysUsed: 0` para seleccionar usuarios que no han usado el servicio, y `credits: { $gt: 4 }` para quedarme solo con los que tienen más de 4 créditos. En la proyección he excluido `_id` para no mostrar el DNI, y he dejado únicamente los campos solicitados. Finalmente, con `.sort({ credits: -1 })` ordeno los resultados de mayor a menor crédito.

El resultado muestra 5 usuarios que cumplen todas las condiciones pedidas. Considero que la consulta responde correctamente al enunciado porque filtra por edad, uso y créditos, excluye el identificador del usuario y presenta los resultados ordenados de forma decreciente por el campo `credits`.

Consulta 2 (10%)

Se desea programar una operación de mantenimiento nocturno. Para ello se necesita identificar la localización de todas aquellas bicicletas que tengan la batería baja (menos del 3%) que estén en estado libre ("free"). Realizar la operación de consulta que recuperaría dichos datos. Mostrar sólo matrícula, nivel de batería y localización (ya que solo esa información es relevante para el equipo que realiza el mantenimiento), omitiendo el resto de los campos. Ordenar por matrícula en orden ascendente.

Para esta segunda consulta se pide identificar las bicicletas que necesitan una operación de mantenimiento nocturno. En concreto, se buscan bicicletas con batería baja, es decir, con menos del 3% de batería, que además estén en estado libre (free). El resultado debe mostrar solo la matrícula de la bicicleta, su nivel de batería y su localización, ordenando por matrícula en orden ascendente.

Antes de plantear la consulta, he comprobado la estructura de un documento de la colección `bikes` para asegurarme de usar los nombres reales de los campos:

```
db.bikes.findOne()
```

En la colección, la matrícula de la bicicleta aparece en el campo `_id`, la batería en `batteryCharge`, el estado en `status` y la localización en `location`.

La consulta ejecutada ha sido:

```
db.bikes.find(
{
  batteryCharge: { $lt: 3 },
  status: "free"
},
{
  _id: 1,
  batteryCharge: 1,
  location: 1
}
).sort({ _id: 1 }).pretty()
```

El resultado obtenido ha sido:

```
{
  "_id" : "456U5D2TJ5",
  "location" : {
    "type" : "Point",
    "coordinates" : [
      -3.669220917070948,
      40.475853725308404
    ]
  },
  "batteryCharge" : 1.01
}
{
  "_id" : "7N9ZURODUR",
  "location" : {
    "type" : "Point",
    "coordinates" : [
      -3.6508716609596834,
      40.41863126378471
    ]
  },
  "batteryCharge" : 2.29
}
{
  "_id" : "AXRQTW25R6",
  "location" : {
    "type" : "Point",
    "coordinates" : [
      -3.6436033758487074,
      40.4452626087907
    ]
  },
  "batteryCharge" : 2.62
}
{
  "_id" : "BXC1B9BDD0",
  "location" : {
    "type" : "Point",
```

```

"coordinates" : [
-3.691698039140152,
40.42898242488213
]
},
"batteryCharge" : 2.63
}
{
  "_id" : "K3HA8U7KS0",
  "location" : {
    "type" : "Point",
    "coordinates" : [
-3.6718214450302566,
40.40731390474217
]
  },
  "batteryCharge" : 0.62
}
{
  "_id" : "XO39RVX4CX",
  "location" : {
    "type" : "Point",
    "coordinates" : [
-3.714498386879484,
40.42449988358074
]
  },
  "batteryCharge" : 2.48
}

```

En esta consulta he usado `batteryCharge: { $lt: 3 }` para seleccionar únicamente las bicicletas con batería inferior al 3%, y `status: "free"` para quedarme solo con las que están disponibles. En la proyección he mostrado únicamente los campos relevantes para el equipo de mantenimiento: matrícula (`_id`), localización (`location`) y nivel de batería (`batteryCharge`). Finalmente, he utilizado `.sort({ _id: 1 })` para ordenar el resultado por matrícula en orden ascendente.

El resultado muestra 6 bicicletas que cumplen las condiciones indicadas. Considero que la consulta responde correctamente al enunciado porque permite identificar de forma directa qué bicicletas libres tienen la batería baja y dónde se encuentran.

Consulta 3 (20%)

¿Cuáles son los daños que pueden tener las bicis y cuántas bicis tienen cada uno de dichos daños?

Para esta tercera consulta se pide identificar cuáles son los daños que pueden tener las bicicletas y cuántas bicicletas tienen cada uno de esos daños. En la colección `bikes`, el campo `damages` es un array, ya que una bicicleta puede tener cero, uno o varios daños registrados.

Para resolverlo, he utilizado una agregación. Primero aplico `$unwind` sobre el array `damages`, de forma que cada daño quede tratado como un elemento independiente. Después agrupo por el tipo de daño y cuento cuántas veces aparece cada uno.

Finalmente, proyecto los campos de salida y ordeno los resultados de mayor a menor número de bicicletas.

```
db.bikes.aggregate([
{
  $unwind: "$damages"
},
{
  $group: {
    _id: "$damages",
    num_bikes: { $sum: 1 }
  }
},
{
  $project: {
    _id: 0,
    damage: "$_id",
    num_bikes: 1
  }
},
{
  $sort: {
    num_bikes: -1,
    damage: 1
  }
}
]).pretty()
```

El resultado obtenido ha sido:

```
{ "num_bikes" : 116, "damage" : "stolenHelmet" }
{ "num_bikes" : 97, "damage" : "scratches" }
{ "num_bikes" : 96, "damage" : "brokenLamp" }
{ "num_bikes" : 95, "damage" : "flatTire" }
```

A partir del resultado, observo que hay cuatro tipos de daños registrados en las bicicletas: stolenHelmet, scratches, brokenLamp y flatTire. El daño más frecuente es stolenHelmet, presente en 116 bicicletas, seguido de scratches, brokenLamp y flatTire.

Considero que la consulta responde correctamente al enunciado porque no cuenta documentos completos directamente, sino que primero descompone el array damages. Esto es necesario porque el campo puede contener varios daños en una misma bicicleta, y cada daño debe contabilizarse de forma independiente.

Consulta 4 (20%)

Obtener los niveles medio, máximo y mínimo de kilómetros recorridos en el subconjunto de bicis que están paradas (es decir, todas aquellas bicis que **no** están en estado “moving”) agrupados por fabricante.

Para esta cuarta consulta se pide obtener los niveles medio, máximo y mínimo de kilómetros recorridos por las bicicletas que están paradas, entendiendo como bicicletas

paradas todas aquellas que no están en estado moving. Además, los resultados deben agruparse por fabricante.

Para resolverlo, he utilizado una agregación sobre la colección bikes. Primero filtro las bicicletas cuyo estado no es moving. Después agrupo por el campo brand y calculo la media, el máximo y el mínimo del campo kms.

```
db.bikes.aggregate([
  {
    $match: {
      status: { $ne: "moving" }
    }
  },
  {
    $group: {
      _id: "$brand",
      avg_kms: { $avg: "$kms" },
      max_kms: { $max: "$kms" },
      min_kms: { $min: "$kms" }
    }
  },
  {
    $project: {
      _id: 0,
      brand: "$_id",
      avg_kms: 1,
      max_kms: 1,
      min_kms: 1
    }
  },
  {
    $sort: {
      brand: 1
    }
  }
]).pretty()
```

El resultado obtenido ha sido:

```
{
  "avg_kms" : 59.06552605920178,
  "max_kms" : 107.49030942191543,
  "min_kms" : 7.259184062386216,
  "brand" : "BH"
}
{
  "avg_kms" : 60.68689499759185,
  "max_kms" : 108.15914145808853,
  "min_kms" : 16.75425190135372,
  "brand" : "Orbea"
}
{
  "avg_kms" : 60.41827882495764,
  "max_kms" : 99.4532174030521,
  "min_kms" : 17.83730570054539,
  "brand" : "Vandor"
}
```

En esta consulta he usado \$match para quedarme únicamente con las bicicletas cuyo estado es distinto de moving. Después, mediante \$group, he agrupado los documentos por fabricante y he calculado las tres métricas solicitadas: kilómetros medios, máximos y mínimos. Finalmente, con \$project he ajustado el formato de salida para mostrar el nombre del fabricante en el campo brand.

A partir del resultado, observo que aparecen tres fabricantes: BH, Orbea y Vandoor. Entre ellos, Orbea presenta la media de kilómetros más alta, con aproximadamente 60.69 km, y también el máximo más alto, con aproximadamente 108.16 km. Considero que la consulta responde correctamente al enunciado porque excluye las bicicletas en movimiento y calcula las métricas pedidas agrupadas por fabricante.

Consulta 5 (20%)

Del conjunto de los últimos 5 viajes de todos los usuarios (representados en el campo `rides`), y para cada pareja de estaciones origen-destino, ¿cuál es el número de viajes realizados entre ambas, así como la media de gasto en viajes entre ellas? Tener en cuenta sólo los viajes en los que el gasto fue superior a 0.5 créditos. Ordenar la lista de mayor a menor en número de viajes y, en caso de igualdad, en orden decreciente de gasto.

Para esta consulta, adjuntar en la respuesta solo los 5 primeros resultados.

NOTA: ha de considerarse la direccionalidad de los trayectos en los resultados. Es decir, considerando por ejemplo las estaciones 10 y 20, los resultados relativos a los viajes con origen en la estación 10 y destino en la estación 20 no han de agruparse con los resultados relativos a los viajes con origen en la estación 20 y destino la estación 10.

Para esta quinta consulta se pide analizar el conjunto de los últimos 5 viajes de todos los usuarios, almacenados en el campo `rides`. Para cada pareja de estaciones origen-destino, se debe calcular el número de viajes realizados y la media de créditos gastados, considerando únicamente los viajes con un gasto superior a 0.5 créditos. Además, los resultados deben ordenarse de mayor a menor número de viajes y, en caso de empate, por gasto medio decreciente. El enunciado también indica que solo deben adjuntarse los 5 primeros resultados.

Antes de resolver la consulta, he comprobado la estructura del campo `rides` en la colección `users`. Cada viaje contiene la estación de origen, la estación de destino, la bicicleta utilizada, la fecha, los kilómetros recorridos y los créditos consumidos.

La consulta ejecutada ha sido:

```
db.users.aggregate([
  {
    $project: {
      rides: { $slice: ["$rides", -5] }
    },
  },
  {
```

```
$unwind: "$rides"
},
{
  $match: {
    "rides.credits": { $gt: 0.5 }
  }
},
{
  $group: {
    _id: {
      from: "$rides.from",
      to: "$rides.to"
    },
    num_trips: { $sum: 1 },
    avg_credits: { $avg: "$rides.credits" }
  }
},
{
  $project: {
    _id: 0,
    from: "$_id.from",
    to: "$_id.to",
    num_trips: 1,
    avg_credits: 1
  }
},
{
  $sort: {
    num_trips: -1,
    avg_credits: -1
  }
},
{
  $limit: 5
}
}).pretty()
```

El resultado obtenido ha sido:

from	to	num_trips	avg_credits
18	10	26	0.5807692307692307
12	43	25	0.612
53	57	25	0.602
2	60	24	0.6095833333333334
59	32	24	0.5962500000000001

En esta agregación, primero utilizo \$project junto con \$slice para quedarme únicamente con los últimos 5 viajes de cada usuario. Después, con \$unwind, descompongo el array rides para tratar cada viaje como un documento independiente. A continuación, filtro con \$match los viajes cuyo gasto en créditos es superior a 0.5.

Posteriormente, agrupo por la pareja origen-destino mediante los campos rides.from y rides.to. Esta agrupación mantiene la direccionalidad del trayecto, ya que un viaje de la estación 18 a la 10 se considera distinto de un viaje de la estación 10 a la 18. En cada grupo calculo el número total de viajes y la media de créditos consumidos.

Finalmente, ordeno los resultados por número de viajes de forma decreciente y, en caso de empate, por gasto medio también de forma decreciente. Con \$limit: 5 muestro únicamente los cinco primeros resultados, tal como pide el enunciado.

El resultado muestra que la pareja origen-destino con más viajes es la formada por la estación 18 como origen y la estación 10 como destino, con 26 viajes y un gasto medio aproximado de 0.58 créditos. Considero que la consulta responde correctamente al enunciado porque trabaja sobre los últimos 5 viajes de cada usuario, filtra únicamente los viajes con gasto superior a 0.5 créditos, mantiene la direccionalidad de los trayectos y devuelve solo los cinco primeros resultados ordenados según los criterios indicados.

Consulta 6 (20%)

Para la resolución de esta consulta, considerar que el campo `nRides` indica cuántos viajes ha hecho ese usuario en total. Sin embargo, el array `rides` contiene información detallada de tan solo los últimos 5 viajes realizados. El primer elemento del array `rides` es el viaje más antiguo, el último elemento del array `rides` es el viaje más reciente.

El usuario identificado por el DNI 75177947Q ha realizado un nuevo viaje, con estas características:

- Distancia: 5.3 kms
- Créditos consumidos: 0.77
- Estación origen: 11
- Estación destino: 24
- Bici: U82PWFSGSG
- Fecha: la actual en el momento de hacer la actualización

Realizar la eliminación del viaje más antiguo (el primero de la lista) y el añadido del nuevo viaje (al final de la lista). Ha de incrementarse también el contador de viajes del usuario, decrementarse su contador de créditos y establecerse como fecha del último uso del servicio la actual en el momento de hacer la actualización.

Así mismo, actualizar los datos de número de viajes y distancia recorrida en la bici identificada por el número de serie anterior.

SUGERENCIA: `new Date()` utilizado en un comando de creación o de actualización como valor para un atributo devuelve un objeto `ISODate` con la fecha actual.

Reflexionar y responder sobre las siguientes cuestiones:

- ¿Cuál es el mínimo de operaciones necesarias para realizar la actualización? Justifica tu respuesta
- En el caso de que no se realice en una sola operación, ¿en qué estado queda la información de la base de datos mientras se realizan las distintas operaciones? ¿Cómo se compara la situación con respecto a las bases de datos relacionales?

Para esta sexta consulta se pide registrar un nuevo viaje realizado por el usuario 75177947Q con la bicicleta U82PWFSGSG. El viaje tiene origen en la estación 11, destino en la estación 24, una distancia de 5.3 km y un coste de 0.77 créditos. Además, se debe actualizar el documento del usuario y el documento de la bicicleta.

Antes de realizar la actualización, he consultado el estado inicial del usuario y de la bicicleta. Para no mostrar todo el array rides, he usado una agregación que muestra el primer viaje almacenado, el último viaje almacenado y los campos relevantes:

```
db.users.aggregate([
  { $match: { _id: "75177947Q" } },
  {
    $project: {
      _id: 1,
      credits: 1,
      nRides: 1,
      lastUse: 1,
      oldestRide: { $arrayElemAt: ["$rides", 0] },
      newestRide: { $arrayElemAt: ["$rides", -1] }
    }
  }
]).pretty()
```

El estado inicial del usuario era:

```
{
  "_id" : "75177947Q",
  "lastUse" : ISODate("2025-05-06T22:04:20Z"),
  "nRides" : 12,
  "credits" : 2.29,
  "oldestRide" : {
    "from" : 48,
    "to" : 14,
    "bike" : "SQSCI9BXR3",
    "date" : ISODate("2025-04-24T11:07:51Z"),
    "kms" : 3.306,
    "credits" : 0.5
  },
  "newestRide" : {
    "from" : 39,
    "to" : 27,
    "bike" : "N63X4LNVIC",
    "date" : ISODate("2025-05-04T14:46:29Z"),
    "kms" : 2.189,
    "credits" : 0.33
  }
}
```

También he consultado el estado inicial de la bicicleta:

```
db.bikes.find(
  { _id: "U82PWFGSG" },
  {
    _id: 1,
    rides: 1,
    kms: 1
  }
).pretty()
```

El resultado inicial era:

```
{ "_id" : "U82PWFGSG", "rides" : 14, "kms" : 37.71391291205472 }
```

A continuación, he guardado la fecha actual en una variable para usar exactamente el mismo valor temporal en `lastUse` y en el nuevo elemento del array `rides`:

```
var fechaViaje = new Date();
```

Después he actualizado el documento del usuario. En una sola operación sobre la colección `users` incremento el contador de viajes, decremento los créditos, actualizo la fecha de último uso y añado el nuevo viaje al final del array `rides`. Además, con `$slice: -5` mantengo únicamente los últimos cinco viajes:

```
db.users.updateOne(
  { _id: "75177947Q" },
  {
    $inc: {
      nRides: 1,
      credits: -0.77
    },
    $set: {
      lastUse: fechaViaje
    },
    $push: {
      rides: {
        $each: [
          {
            from: 11,
            to: 24,
            bike: "U82PWFGSG",
            date: fechaViaje,
            kms: 5.3,
            credits: 0.77
          }
        ],
        $slice: -5
      }
    }
  }
);
```

La respuesta de MongoDB ha sido:

```
{ "acknowledged" : true, "matchedCount" : 1, "modifiedCount" : 1 }
```

Después he actualizado el documento de la bicicleta, incrementando en 1 su número de viajes y sumando 5.3 km a los kilómetros acumulados:

```
db.bikes.updateOne(
  { _id: "U82PWFGSG" },
  {
    $inc: {
      rides: 1,
      kms: 5.3
    }
  }
);
```

La respuesta de MongoDB ha sido:

```
{ "acknowledged" : true, "matchedCount" : 1, "modifiedCount" : 1 }
```

Finalmente, he vuelto a consultar el usuario para validar la modificación:

```
db.users.aggregate([
  { $match: { _id: "75177947Q" } },
  {
    $project: {
      _id: 1,
      credits: 1,
      nRides: 1,
      lastUse: 1,
      oldestRide: { $arrayElemAt: ["$rides", 0] },
      newestRide: { $arrayElemAt: ["$rides", -1] }
    }
  }
]).pretty()
```

El resultado después de la actualización ha sido:

```
{
  "_id" : "75177947Q",
  "lastUse" : ISODate("2026-06-11T15:30:40.082Z"),
  "nRides" : 13,
  "credits" : 1.52,
  "oldestRide" : {
    "from" : 61,
    "to" : 31,
    "bike" : "4615Y5TXTM",
    "date" : ISODate("2025-04-27T14:07:24Z"),
    "kms" : 3.927,
    "credits" : 0.59
  },
  "newestRide" : {
    "from" : 11,
    "to" : 24,
    "bike" : "U82PWFSGSG",
    "date" : ISODate("2026-06-11T15:30:40.082Z"),
    "kms" : 5.3,
    "credits" : 0.77
  }
}
```

También he validado el documento de la bicicleta:

```
db.bikes.find(
  { _id: "U82PWFSGSG" },
  {
    _id: 1,
    rides: 1,
    kms: 1
  }
).pretty()
```

El resultado después de la actualización ha sido:

```
{ "_id" : "U82PWFSGSG", "rides" : 15, "kms" : 43.013912912054714 }
```

La validación confirma que la actualización se ha realizado correctamente. En el usuario, `nRides` ha pasado de 12 a 13, los créditos han pasado de 2.29 a 1.52 y `lastUse` se ha actualizado con la fecha del nuevo viaje. Además, el array `rides` sigue manteniendo solo cinco elementos: se ha eliminado el viaje más antiguo y se ha añadido el nuevo viaje al final.

En la bicicleta, el número de viajes ha pasado de 14 a 15 y los kilómetros acumulados han pasado de 37.71391291205472 a 43.013912912054714, que corresponde a sumar los 5.3 km del nuevo viaje.

El mínimo de operaciones de actualización necesarias es de dos: una operación sobre la colección `users` y otra sobre la colección `bikes`. En el documento del usuario, una sola operación permite actualizar varios aspectos a la vez mediante `$inc`, `$set` y `$push` con `$slice`. Sin embargo, la bicicleta está almacenada en otra colección, por lo que requiere una segunda operación para actualizar su contador de viajes y sus kilómetros acumulados.

Como no se realiza todo en una única operación atómica conjunta sobre los dos documentos, durante el intervalo entre ambas actualizaciones la base de datos podría quedar en un estado intermedio. Por ejemplo, si se actualizara el usuario pero fallara la actualización de la bicicleta, el usuario ya tendría registrado el nuevo viaje, pero la bicicleta todavía conservaría el número de viajes y kilómetros anteriores. En ese caso, la información quedaría desincronizada hasta corregir manualmente la operación o usar una transacción multidocumento.

En una base de datos relacional, esta actualización normalmente implicaría varias sentencias sobre tablas distintas, por ejemplo una inserción en una tabla de viajes y varias actualizaciones en las tablas de usuarios y bicicletas. Lo habitual sería ejecutar todas esas operaciones dentro de una transacción para garantizar la consistencia. En MongoDB, las actualizaciones sobre un único documento son atómicas, lo que simplifica la actualización del usuario porque la información de sus últimos viajes está embebida en su propio documento. Sin embargo, como también hay que actualizar el documento de la bicicleta, sigue siendo necesario coordinar más de una operación si se quiere mantener la consistencia completa del sistema.